What Should Count as a Token?
A Comprehensive Study of Data Representation for Foundation Models

TODO

Abstract

Draft suggestion: Foundation models operate on finite collections of addressable units, yet the choice of these units often receives less study than the architecture that processes them. Tokenization determines unit boundaries, representation scale, retained information, explicit relations, and computational allocation. These choices are especially important in science, where measurement units may differ from the structural or functional units required by a task. We define a tokenizer as a three-module system that constructs reusable parameters, maps an input to a token representation, and maps that representation to a reconstruction. This definition covers algorithmic, neural, and hybrid tokenizers while separating token formation from the downstream foundation model.

Draft suggestion: We study how these methods define tokens for different AI modalities and scientific domain objects, identifying the component tokenizers used to represent individual data sources or objects. We also study system-level tokenization for complex settings that combine multiple modalities, domain objects, spatial or temporal scales, and coupled observations and actions.

Draft suggestion: Across these levels, we examine the trade-offs among coverage, compression, reconstruction, task information, scientific constraints, and model compatibility. We conclude with design principles, evaluation protocols, and a research agenda for adaptive, multiscale, and reusable tokenizers that support future foundation models in prediction, generation, and interaction.

Draft suggestion: Note: Please add your references in a separate bib file in the bib folder. Please reuse the citation if warnings are raised; don't worry about the same paper with different cite-ids

Contents

1 Introduction **4**

1.1 Motivation, Scope, and Contributions 4

1.2 From Compression to Intelligence 4

1.3 Observations and Insights 4

2 Tokenizer Systems: Definition and Method Taxonomy **4**

2.1 A Tokenizer as a Three-Module System 5

2.2 Algorithmic Tokenizers 6

2.3 Neural and Hybrid Tokenizers 6

3 Data Representation and Tokenization across AI Modalities **7**

3.1 Sequences 7

3.2 Images . . . 7
3.3 Video . . . 7
3.4 Audio . . . 7
3.5 Graphs . . . 8
3.6 3D Geometry . . . 8
3.7 Tables . . . 8
3.8 Time Series and Signals . . . 8

4 Data Representation and Tokenization by Domain Object 8
4.1 Language and Formal Objects . . . 8
4.1.1 Natural Language . . . 9
4.1.2 Code and Mathematics . . . 9
4.2 Music and Sound Events . . . 9
4.3 Molecular and Material Objects . . . 9
4.3.1 Nucleic Acids . . . 9
4.3.2 Proteins and Peptides . . . 10
4.3.3 Small Molecules and Reactions . . . 10
4.3.4 Synthetic Polymers . . . 11
4.3.5 Crystals . . . 12
4.4 Biomedical Observations . . . 16
4.4.1 Gene Expression Profiles . . . 16
4.4.2 Cell and Tissue Morphology . . . 16
4.4.3 Clinical Records and Measurements . . . 16
4.5 Physical and Robotic Objects . . . 16
4.5.1 Physical Fields . . . 16
4.5.2 Scenes and Geographical Regions . . . 16
4.5.3 Robot Actions . . . 17
4.6 Recommendation Data . . . 17

5 Tokenization for Complex Systems 17
5.1 Multimodal Language Models . . . 17
5.2 Biomolecular Complexes . . . 17
5.3 Cellular Perturbation Systems . . . 18
5.4 World Models . . . 18
5.5 User Behavior Systems . . . 18

6 Tokenizer Design and Evaluation 18

6.1 Core Properties of a Tokenizer System 18
6.2 Task and Scientific Design Principles 19
6.3 Benchmarks and Comparisons 19

7 Future Directions **19**
7.1 Evaluating Tokenizers with and without Foundation Models 19
7.2 Designing Tokenizers for Foundation Models 20
7.3 Co-designing Tokenizers and Foundation Models 20

8 Conclusion **20**

1 Introduction

Draft suggestion: Extend the related-work coverage by searching for unit formation under segmentation, compression, patching, pooling, quantization, motif discovery, grammar induction, discretization, and state abstraction. Prioritize papers that introduce or directly evaluate tokenization. Retain reuse-only systems when they have broad impact, document a major public model, or supply a comparison that is missing from the review.

1.1 Motivation, Scope, and Contributions

Draft suggestion: State the intended readers, the inclusion rule, and the three contributions in terms that can be checked against the catalog. Keep examples representative rather than exhaustive. Present AI modality and domain object as two views of component tokenizers, then present complex systems as the composition of those components across modalities, objects, scales, or time.

1.2 From Compression to Intelligence

Draft suggestion: Trace the technical line from coding and dictionary compression to subword learning, vector quantization, fixed patches, and learned unit selection. Emphasize which ideas changed tokenizer training, encoding, or decoding. Avoid presenting the history as a sequence of foundation-model architectures.

1.3 Observations and Insights

Draft suggestion: Tokenizer systems are often compared only through their output units. Test whether reported conclusions change after separating tokenizer training, encoding, and decoding. Useful cases include subword vocabularies with different segmentation rules, visual codebooks with different decoders, and molecular vocabularies with incomplete inverse mappings. Keep this observation provisional until each case has direct evidence.

Draft suggestion: Scientific token identity often depends on relations and scale. Compare atoms with motifs, residues with structure units, genes with cell states, and grid cells with adaptive regions. Record when geometry, symmetry, experimental condition, or task changes the useful unit. Include cases where fixed small units remain the stronger choice.

Draft suggestion: Compression, reconstruction, and scientific utility can disagree. Look for tokenizers that reduce sequence length or reconstruction error while losing rare events, valid composition, intervention response, or transfer information. Compare the tokenizer training objective with every later evaluation target before making a general claim.

2 Tokenizer Systems: Definition and Method Taxonomy

Draft suggestion: Use the three-module definition as the common comparison frame for every method family. Classify only the operations that determine token units, assignments, or reconstruction. Keep embedding layers, foundation-model backbones, and task heads outside the tokenizer unless they change one of these operations.

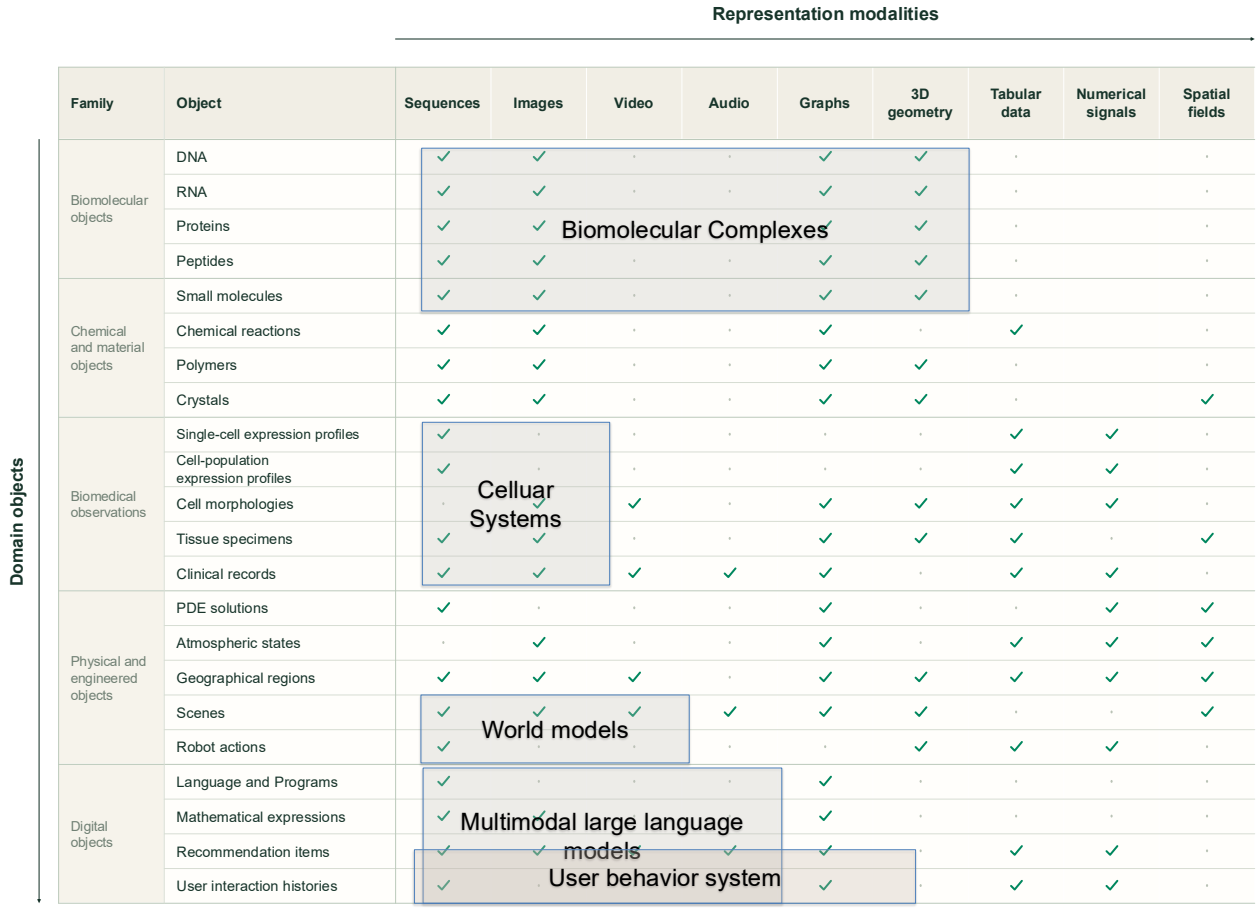


Figure 1: Overview of the work: We organize the review into two major dimensions (representations and objects). (The current tick is tentative; feel free to let me know your suggestion and to remove or add new ticks based on your survey.)

2.1 A Tokenizer as a Three-Module System

Draft suggestion: Keep the definition independent of modality and downstream architecture. Explain the roles of `train`, `encode`, and `decode` through BPE, VQ-VAE, and fixed image patches. State the reconstruction and equivalence rule for lossy or structured data, then use the definition to separate full tokenizers from methods that specify only one module.

Let $\mathcal{X}$ denote an input space and $\mathcal{Z}$ a token space. We define a tokenizer system as

$$\mathcal{T} = \big(\text{train}, \text{encode}, \text{decode}\big), \tag{1}$$

with operations

$$\theta = \text{train}(\mathcal{D}), \qquad Z = \text{encode}_\theta(x), \qquad \widehat{x} = \text{decode}_\theta(Z). \tag{2}$$

Here, $\mathcal{D}$ is the data used to construct or fit the tokenizer, $x \in \mathcal{X}$ is an input, $Z \in \mathcal{Z}$ is its token representation, and $\widehat{x} \in \mathcal{X}$ is the reconstructed input. We use θ for all reusable quantities returned by training. These quantities may include a vocabulary, merge table, or codebook. They may also include deterministic rules or neural parameters. A predefined tokenizer is the special case $\text{train}(\mathcal{D}) = \theta_0$ for every $\mathcal{D}$. For a structured input, the encoded representation can be written as

$$Z = \big((z_i)_{i=1}^{n}, R\big), \tag{3}$$

where z_i is one token and R records any relations required to interpret the token collection. Sequence order can define R implicitly. Graph edges, spatial coordinates, or hierarchical links may define it explicitly. A unit qualifies as a token when a later model can process it separately for masking, prediction, or routing. The decoder recovers what information the tokenizer preserves. Exact recovery requires

$$\text{decode}_\theta\big(\text{encode}_\theta(x)\big) = x. \tag{4}$$

Some domains have several records for the same object. In such cases, recovery may be defined up to a stated equivalence, such as graph isomorphism, geometric symmetry, or text normalization. Continuous data usually require an approximation for reconstruction rather than equality. Therefore, the reconstruction target and its comparison rule are part of the tokenizer definition.

We categorize tokenizers into three classes based on how their training algorithms are defined: (1) algorithmic tokenizers, (2) neural tokenizers, and (3) hybrid tokenizers. Algorithmic tokenizers use non-neural training algorithms, which can be either predefined or data-driven, whereas neural tokenizers rely on generative models and quantization techniques.

Byte pair encoding (BPE) is an example of an algorithmic tokenizer. Its training function constructs a vocabulary and an ordered merge table from corpus statistics. The encoding function applies the learned merges to a new sequence, while the decoding function concatenates the resulting symbols.

Another example of an algorithmic tokenizer is image patching, as used in the Vision Transformer (ViT) [1]. In this predefined tokenizer, θ_0 specifies the patch size and ordering. In contrast, vector-quantized variational autoencoders (VQ-VAEs) are examples of neural tokenizers [2] and have been adapted to continuous and non-discrete data. In the following subsections, we define these three classes in detail.

2.2 Algorithmic Tokenizers

BPE-like

Draft suggestion: Develop this subsection around the three sources of unit formation rather than a long list of algorithms. For each family, compare what `train` estimates, how `encode` resolves competing segmentations, what `decode` recovers, and which choices introduce randomness. Select a small number of examples from text, signals, graphs, and scientific structures.

2.3 Neural and Hybrid Tokenizers

Draft suggestion: Ask section authors to identify the last operation that changes token identity or membership. Use that boundary to distinguish the tokenizer from embedding layers, backbones, task heads, and generators. Compare neural systems by their learned token operation and compare hybrid systems by how the algorithmic and neural stages are composed.

A vector-quantized variational autoencoder (VQ-VAE) gives a neural example [2]. The training function jointly learns a neural network encoder f_ϕ, a codebook $C = \{c_j\}_{j=1}^{K}$, and a neural network decoder g_ψ. For each local feature, the encoding function assigns

$$k_i = \arg\min_{1 \le j \le K} \|f_\phi(x)_i - c_j\|_2, \qquad \widehat{x} = g_\psi(c_{k_1}, \ldots, c_{k_n}). \tag{5}$$

The tokens are the code indices $(k_i)_{i=1}^{n}$, and decoding is generally approximate. The codebook is part of θ.

3 Data Representation and Tokenization across AI Modalities

Draft suggestion: Write this section for machine learning researchers and organize it by representation form. Use general AI tasks and models to compare token units, relations, training objectives, decoding targets, token rates, and computational cost. Mention scientific examples such as SMILES strings, protein sequences, molecular graphs, or physical trajectories only briefly, then refer readers to Section 4 for their object-specific meaning and tests. This section should explain transferable tokenization methods rather than repeat the scientific-object review.

3.1 Sequences

[zhenyu]

Draft suggestion: Representative work: Sennrich et al. [3]. Treat a sequence as an ordered representation and compare characters, bytes, subwords, words, and syntax-aware units. Use text and code as the main AI cases. Discuss coverage, sequence length, reversibility, multilingual behavior, and syntactic structure. Mention SMILES, nucleic-acid sequences, and protein sequences only as short transfer examples, then refer readers to Section 4.

3.2 Images

Draft suggestion: Representative work: Dosovitskiy et al. [1] and Van Den Oord et al. [2]. Compare pixels, fixed patches, regions, object-level units, and latent visual codes for recognition and generation. Separate token boundaries from token values and compare reconstruction, token rate, spatial resolution, and compatibility with vision backbones. Treat scientific grids as images only when their row-and-column layout defines the model input; discuss their scientific meaning in Section 4.

3.3 Video

Draft suggestion: Use video tokenization work that studies frames, tubelets, motion-aware units, and discrete visual codes. Explain how temporal continuity changes boundary formation, token rate, and decoding compared with images. Discuss scientific video only briefly here and refer object-specific questions about calibrated intensity, event timing, and identity across frames to Section 4.

Video tokenization begins by determining what constitutes an addressable unit within a temporally evolving visual signal. A video can be organized into individual frames, spatial patches within frames, or tubelets spanning multiple frames. These choices naturally establish geometric boundaries: frame-wise patches separate spatial content at each sampled time, whereas tubelets group a spatial region across a short temporal interval, as illustrated by ViViT.

3.4 Audio

Draft suggestion: Representative work: Baevski et al. [4]. Compare waveform samples, frames, learned speech units, and neural codec codes while separating linguistic content from acoustic information. Cover speech, music, and sound events as distinct uses of the same representation modality. Keep generic sampled measurements in the time-series and signals subsection.

3.5 Graphs

Draft suggestion: Representative work: Galkin et al. [5]. Compare nodes, edges, paths, neighborhoods, motifs, and learned graph units. State whether adjacency belongs to the token representation or remains external relation data. Discuss permutation, structural roles, graph reconstruction, and vocabulary transfer. Use molecular, crystal, protein, and cell graphs only as short examples before referring readers to Section 4.

3.6 3D Geometry

Draft suggestion: Representative work: Yu et al. [6]. Compare points, voxels, mesh elements, local geometric patches, and object-level units when coordinates, shape, or spatial neighborhoods determine token identity. Discuss sampling density, rigid transformations, local frames, and geometric reconstruction. Treat a mesh as a graph when combinatorial adjacency defines the units, and treat it as 3D geometry when coordinates and shape define them.

3.7 Tables

Draft suggestion: Compare cells, columns, rows, records, and learned groups as candidate table tokens. Explain how schema, data type, missing values, and row or column order affect token identity. Treat sets as order-free inputs within this discussion when their elements share the same representation rules. Refer gene-expression matrices, clinical measurements, and interaction histories to their domain-object subsections.

3.8 Time Series and Signals

Draft suggestion: Representative work: Lin et al. [7]. Compare samples, windows, events, transformed coefficients, and quantized measurements for forecasting, anomaly detection, and numerical sequence modeling. Distinguish time-indexed measurements from video, where spatial appearance remains central, and from audio, where waveform and acoustic structure define the task. Mention molecular trajectories, physical simulations, and Earth dynamics only to show method transfer, then refer their scientific tests to Section 4.

4 Data Representation and Tokenization by Domain Object

Draft suggestion: Organize this section by what is represented and use the modality methods in Section 3 as technical background. Use the six families below only to structure the review. Place each concrete object in a subsubsection. Avoid repeating BPE, patching, graph pooling, vector quantization, or signal segmentation. Explain instead how object identity, accepted equivalences, known relations, measurement conditions, and validation rules change the tokenization problem. Refer systems that combine several objects or modalities to Section 5.

4.1 Language and Formal Objects

[zhenyu]

Draft suggestion: Use this family to compare tokenization for natural language, source code, and mathematical expressions without treating them as one representation. The sequence methods in Section 3 provide the shared background. Focus here on the linguistic, syntactic, and formal constraints that determine valid boundaries and decoding.

4.1.1 Natural Language

Draft suggestion: Representative work: Sennrich et al. [3]. Discuss words, characters, bytes, and learned subwords through input coverage, linguistic boundaries, multilingual transfer, normalization, and exact decoding. Separate the tokenizer from the language model that consumes its output.

4.1.2 Code and Mathematics

Draft suggestion: Compare lexical tokens, learned subwords, syntax-tree units, and serialized mathematical expressions. Ask how a tokenizer handles identifiers, whitespace, formal grammar, variable names, and equivalent expressions. Use sequence and graph tokenization from Section 3 as background, then focus on syntactic validity and executable or symbolic equivalence.

1. grammarvae, DIGGED

4.2 Music and Sound Events

Draft suggestion: Keep this family separate from natural language because musical structure and non-speech events use different units and decoding targets. Build on the audio methods in Section 3, then compare event tokens, note-like symbols, acoustic codec codes, and their temporal hierarchy.

Draft suggestion: Discuss symbolic music, acoustic music, and non-speech events through timing, pitch or event identity, polyphony, hierarchy, and waveform reconstruction. State whether tokens represent semantic events, acoustic segments, or codec codes, because these choices support different prediction and generation tasks.

4.3 Molecular and Material Objects

Draft suggestion: This family groups objects whose token definitions depend on chemical identity, bonding, geometry, composition, or periodicity. Keep the five object subsubsections distinct. Compare them through valid transformations, reconstruction targets, scale, and physical constraints rather than through one shared chemical vocabulary.

4.3.1 Nucleic Acids

Draft suggestion: Representative work: Dalla-Torre et al. [8]. Sequence methods overlap with Section 3; summarize them briefly and focus here on bases, codons, regulatory regions, reverse complements, reading frames, mutations, and transfer across genomes. Move joint nucleic-acid and protein systems to Section 5.

Representative works:

1. dna language models (hyena, evo, alphagenome, etc.)

2. Taehyun Oh's works (inspired from h-net)
3. cite his related works
4. vq-vae works

4.3.2 Proteins and Peptides

Draft suggestion: Representative work: Rives et al. [9]. This object overlaps with sequences, graphs, 3D geometry, and time series in Section 3. Focus on residues, motifs, domains, structural units, conformational states, noncanonical residues, and biological function. Include peptides when residue sequence, conformation, or biological function defines the problem. Discuss protein complexes and multiscale biology in Section 5.

1. esm family (sequence)
2. vq-vae, geobpe (structure)

4.3.3 Small Molecules and Reactions

Draft suggestion: Representative work: Jin et al. [10]. Sequences, graphs, 3D geometry, spectra, and trajectories overlap with Section 3; focus here on molecular equivalence, stereochemistry, valence, fragments, reaction centers, chemical validity, and inverse reconstruction. Treat reactions as transformations among molecular objects. Move biomolecular assemblies with proteins or nucleic acids to Section 5.

1. tokenize molecules procedurally as reactions (synnet, chemprojector, synformer, synthesisnet)
2. tokenize molecules using rules that align with reactions and generate synthesizable molecules (randomwalk, FMG, alphagrammar, mclm)

4.3.4 Synthetic Polymers

Many synthetic polymer samples comprise chains with different lengths, sequences, or branching patterns. A repeat unit specifies local chemistry, but it does not fully specify such a sample. Polymer tokenization therefore depends on whether the input describes an ideal repeating structure, an individual chain, or an ensemble of chains [11, 12]. Here, 1D refers to string representations and 2D to chemical connectivity graphs, following the sequence and graph methods in Section 3. In both cases, the choice of units determines how repeat chemistry, connections between units, and sample-level information enter the model. Table 1 summarizes representative approaches.

1D representations. For linear polymers, polymer SMILES (PSMILES) describes a repeat unit with two wildcard connection sites, often written as `[*]`. The wildcards mark backbone continuation rather than the chemical end groups of a finite chain. One difficulty is that the same ideal polymer admits several descriptions: `[*]CO[*]` and `[*]COCO[*]`, for example, encode the same periodic connectivity using different repeat lengths. polyBERT addresses this redundancy through polymer-specific canonicalization, including repeat reduction and normalization of the repeat boundary, before applying a 265-token SentencePiece vocabulary [13]. The resulting sequence represents the canonical PSMILES string. The fingerprint subsequently obtained by averaging token embeddings cannot be uniquely mapped back to that string. Thus, the information retained by the string tokenizer must be distinguished from that retained by the learned fingerprint.

TransPolymer extends the input beyond repeat chemistry by combining SMILES with available descriptors, including component ratios, chain arrangement, molecular weight, and degree of polymerization [14]. Its chemically informed tokenization keeps multi-character elements such as `Si` together, assigns tokens to discretized numerical values, and uses descriptor-specific markers for missing values. This allows samples with the same repeat chemistry to be distinguished when the relevant descriptors are available. The numerical precision and completeness of the representation depend on how those descriptors are recorded and discretized.

Generative models also need a representation that can be decoded into a polymer structure. polyBART uses pseudo-polymer SELFIES (PSELFIES), obtained by closing and canonicalizing a repeat-unit graph, reopening it, and introducing astatine atoms as proxies for the two connection sites [15]. SELFIES symbols then provide input units compatible with a pretrained molecular language model. The proxy atoms carry a polymer-specific meaning: polyBART checks that a decoded structure contains exactly two singly bonded At termini. This check is separate from molecular validity, and neither criterion alone establishes that a proposed polymer can be synthesized.

These repeat-unit strings describe deterministic connectivity. BigSMILES instead describes ensembles through stochastic objects containing repeat-unit alternatives and bonding descriptors that specify permitted connections [11]. Its syntax accommodates statistical copolymers and branched structures without enumerating individual chains. The order of units in a stochastic list does not specify their order along a chain. For tokenization, the meaningful boundaries include repeat units, stochastic objects, and labeled bonding sites; losing the labels or object boundaries changes the permitted connectivity. BigSMILES supplies this representation language, rather than a particular token vocabulary. Its original formulation specifies possible structures without assigning a probability to each realization, so additional statistics are needed to describe a quantitative ensemble.

2D representations. Chemical graphs expose connectivity directly, with atom or bond attributes as local units and adjacency as explicit relation data. Polymer graphs must also describe connections across repeat boundaries. Aldeghi and Coley represent molecular ensembles using directed edges weighted by connection probabilities, together with the relative abundances of the constituent units [12]. Their weighted directed message-passing neural network (wD-MPNN) uses these quantities to distinguish copolymer composition and architecture; average degree of polymerization is additionally supplied in their phase-prediction task. This representation retains an ensemble average, but not its full heterogeneity. The authors identify specific ambiguities: gradient and block copolymers can share the same average connectivity, while samples with the same average chain length can have different dispersities.

GRIN examines a related problem: the embedding of a polymer can change when its graph is constructed with a different number of repeats [16]. It combines repeat-unit augmentation with max-aggregation graph learning inspired by maximum spanning trees. The augmented graphs share the same property label, so each repeat-count variant is trained against the same target. Its invariance applies to these graph variants; physical chain length remains a separate sample variable. The reported copolymer experiments cover block and alternating

architectures; they do not establish generality to arbitrary stochastic sequences. Both GRIN and wD-MPNN learn features over prescribed atom and bond graphs, rather than a discrete vocabulary of polymer motifs.

Implications for token design. Repeat units or larger motifs offer possible coarser tokens, provided their attachment sites and inter-unit bonds remain specified. A useful polymer representation must be stable under equivalent choices of repeat boundary while retaining differences in composition, sequence statistics, and chain architecture. Molecular-weight distribution and processing or measurement conditions require additional fields when they matter to the task. Reconstructing a repeat-unit string is therefore a narrower objective than reconstructing a sample specification. Comparisons should assess both: whether equivalent descriptions yield consistent representations, and whether chemically or statistically different samples remain distinguishable. Alternative descriptions of the same polymer must also remain within one dataset split, as in the augmentation protocol used by TransPolymer [14].

Table 1: Representative polymer representations. The table distinguishes input units from the information retained by the representation.

Representation / example	Input units and relations	Polymer information and limitations
PSMILES / polyBERT [13]	SentencePiece tokens over canonical PSMILES, including connection-site and syntax symbols.	Canonical repeat chemistry; the pooled fingerprint is not uniquely decodable.
Descriptor strings / TransPolymer [14]	Chemical symbols, discretized descriptor values, separators, and missing-value markers.	Repeat chemistry and supplied sample descriptors; numerical resolution and missing data limit detail.
PSELFIES / polyBART [15]	SELFIES symbols, with At proxies for repeat-unit connection sites.	Repeat-unit candidates; valid molecular outputs may still have invalid polymer endpoints.
BigSMILES [11]	Repeat-unit alternatives, stochastic objects, and labeled bonding descriptors.	Permitted ensemble connectivity; realization probabilities need further specification.
Ensemble graph / wD-MPNN [12]	Atom and directed-bond attributes, connection weights, and unit fractions.	Average composition and architecture; full dispersity is not represented.
Augmented graphs / GRIN [16]	Atom/bond graphs joined across repeat boundaries; repeat count varies during training.	Embeddings stable across repeat-count variants with shared labels; physical chain-length dependence is not modeled by this invariance.

4.3.5 Crystals

A crystal tokenizer maps an infinite periodic arrangement to finite, addressable units. An ordered structure can be written as $x = (L, A, F)$, with lattice L, element identities $A = (a_i)_{i=1}^{n}$, and fractional coordinates $F = (f_i)_{i=1}^{n}$; its sites are $L(f_i + k)$ for $k \in \mathbb{Z}^3$. Atom permutations, origin shifts, and equivalent cell choices change this record while preserving the crystal. Following the survey's three-module definition, we compare the represented object, token units and vocabulary, construction of tokenizer parameters, and the algorithms that encode and decode a structure. Table 2 applies these questions to representative methods.

Representation. Crystal representations retain different parts of composition, geometry, connectivity, and symmetry. CGCNN represents atoms and periodic neighbors as a multigraph for property prediction [17]; coGN supports symmetry-reduced graphs over asymmetric-unit sites and their periodic images [18]. DiffCSP retains the lattice and fractional coordinates as explicit geometric variables [19]. CrystaLLM and Mat2Seq serialize geometric structure records [20, 21], whereas SLICES stores composition and periodic connectivity, and SLICES-PLUS appends spatial symmetry operations [22, 23].

Symmetry reduction replaces repeated atoms with orbit representatives. CrystalFormer, PLaID++, and MatraGenoa retain representative-site coordinates and lattice parameters, allowing orbit expansion into a complete cell [24–26]. Coordinate-free Wyckoff descriptions instead group geometric realizations into one decorated symmetry prototype. Goodall et al. use this coarse graining for stability prediction [27]; WyFormer uses it for

generation [28]. CDVAE learns a continuous latent representation from which a neural decoder reconstructs the structure [29]. These choices determine whether decoding recovers an explicit geometry, infers one from connectivity, or supplies a member of a prototype family.

Token units and vocabulary. Crystal sites, symmetry orbits, and graph edges become addressable units through vocabulary lookup or continuous feature maps. CrystaLLM defines a symbolic vocabulary of CIF tags, element and space-group symbols, digits, and punctuation. Mat2Seq uses keywords and numerical symbols over a reduced-cell description. PLaID++ passes Wyckoff-field text to a pretrained language tokenizer, so one crystallographic site can occupy several text tokens. SLICES uses element symbols, node indices, and integer cell-translation labels; SLICES-PLUS adds symbols for rotation matrices and fractional translations. Their vocabulary entries describe syntax, chemistry, and relations at different scales.

WyFormer binds an element, site-symmetry category, and enumeration index into a site token, accompanied by a global space-group token. The enumeration distinguishes Wyckoff positions with the same site symmetry within a space group. CrystalFormer uses discrete element and Wyckoff categories together with continuous positional and lattice variables. Matra-Genoa likewise combines categorical fields with continuous geometry, using Gaussian embeddings for continuous values. Thus, a finite vocabulary may cover only part of a crystal record. Atom and edge units in graph methods carry continuous geometric features, while CDVAE uses a continuous latent space rather than a finite codebook.

Tokenizer training algorithm. Predefined tokenizers construct θ_0 from crystallographic tables, neighborhood rules, serialization grammar, and numerical precision. CGCNN graph construction, Mat2Seq cell reduction, SLICES graph serialization, and Wyckoff-site identification follow such rules. They require no corpus-fitted merge table. CrystalFormer, WyFormer, and Matra-Genoa train embeddings and generative models over prescribed fields; their model training does not learn the field boundaries. PLaID++ inherits its language vocabulary and segmentation rules from the base model, then fine-tunes the model on serialized crystals.

CDVAE learns its encoding and reconstruction maps jointly. A periodic graph encoder predicts a latent Gaussian distribution; training combines a KL penalty with objectives for atom count, composition, lattice parameters, and denoising atom types and positions. Reconstruction losses shape the information stored in the latent. SLICES occupies another point in the design space: encoding uses fixed graph rules, while its reconstruction pipeline reuses a pretrained interatomic potential after geometric optimization. Tokenizer training can therefore consist of fixed parameter construction, reuse of an existing tokenizer or decoder, or joint fitting of an encoder–decoder pair.

Encoding algorithm. For graph inputs, encoding selects periodic neighbors and constructs atom and edge attributes. CGCNN expands neighbor distances into Gaussian features in its released implementation; coGN supports reduction to asymmetric-unit nodes, retains image transformations and multiplicities, and builds nested line-graph relations for local geometry. DiffCSP extracts (L, A, F) directly, with fractional coordinates treated periodically. CDVAE then adds a learned graph-to-latent mapping when a compressed latent is the output of encoding.

Sequence encoders additionally choose a cell, an ordering, and a numerical representation. CrystaLLM standardizes a CIF and applies lexical token lookup. Mat2Seq performs Niggli reduction, selects an origin, orders irreducible sites, and serializes their geometry and symmetry information; its construction targets rigid and periodic invariance. SLICES identifies neighbors with EconNN and writes a labeled quotient graph. SLICES-PLUS adds a symbolic sequence of symmetry operations. Wyckoff encoders identify orbits and attach elements: CrystalFormer orders Wyckoff letters and representative coordinates, WyFormer maps positions to site-symmetry and enumeration categories, and PLaID++ refines a cell before writing its Wyckoff text. Matra-Genoa serializes categorical and continuous fields into an invertible mixed sequence. Ordering and symmetry tolerance enter at this stage, before a generative model predicts any tokens.

Decoding algorithm. Explicit geometric records support parsing and coordinate recovery. CrystaLLM concatenates lexical tokens into CIF text for parsing. Mat2Seq recovers lattice parameters and expands irreducible sites using symmetry operations. CrystalFormer and Matra-Genoa recover representative geometry and apply the corresponding Wyckoff operations; PLaID++ converts language tokens to text, parses its fields, and expands sites through PyXtal. Stored precision determines geometric resolution: Mat2Seq rounds real values to four decimal places, while PLaID++ uses two for lattice parameters and three for fractional coordinates.

Topology and latent representations require additional geometry construction. SLICES' SLI2Cry routine embeds the periodic graph, optimizes chemically informed geometry, and refines it with an interatomic potential; SLICES-PLUS augments the graph representation with symmetry information. WyFormer decodes to a protostructure, after which PyXtal and geometry optimization or a separate geometric model supply coordinates and lattice values. CDVAE predicts global structure attributes from its latent and uses Langevin denoising to reconstruct atoms and positions. CGCNN and coGN specify property-oriented graph encodings without a structure-reconstruction decoder; DiffCSP's state already contains explicit geometry, and its reverse diffusion learns a distribution over that state. Figure 2 contrasts these encoding and decoding paths.

Coverage and evaluation. Defects change the object that encoding must preserve. A periodic supercell fixes both defect concentration and repeat distance; symmetry lowering increases the number of independent sites and reduces orbit-based compression. Partial occupancy describes an average disordered structure, while CrystaLLM's training data contain ordered structures [20]. Element coverage, occupancy conventions, cell size, and symmetry tolerance therefore enter the input definition.

Tokenizer evaluation can follow the same module boundaries. For `train`, report fitted or reused components and the data used to obtain them. For `encode`, measure token and relation counts, numerical precision, cost, and consistency under equivalent cells, origin shifts, and atom permutations. For `decode`, measure structural matching and geometric error at a stated tolerance, separating record recovery from geometry inferred by optimization or a learned decoder. Changes in composition, defects, or free coordinates should remain distinguishable when they belong to the target. Diffraction supplies a linked measurement for evaluating reconstructed crystals under specified experimental conditions.

Table 2: Features of representative crystal tokenizations. Checkmarks indicate neural latent encoding (N), explicit complete geometry (G), and explicit crystallographic symmetry information (S). Type: D, discrete; M, mixed discrete–continuous; C, continuous latent.

Method	Units	Type	N	G	S	Tokenizer training	Reconstruction
CGCNN [17]	Atoms, periodic edges	M	–	–	–	Fixed graph rules	Not specified
coGN [18]	Reduced sites, image edges	M	–	–	✓	Fixed graph rules	Not specified
DiffCSP [19]	Elements, coordinates, lattice	M	–	✓	–	Fixed fields	Explicit state
CDVAE [29]	Crystal latent	C	✓	–	–	Joint encoder–decoder	Learned, approximate
CrystaLLM [20]	CIF symbols and digits	D	–	✓	✓	Fixed vocabulary	CIF parsing
Mat2Seq [21]	Reduced-cell and site fields	D	–	✓	✓	Fixed rules	Symmetry expansion
SLICES [22]	Elements, indices, cell shifts	D	–	–	–	Fixed encoder; reused potential	Graph embedding and optimization
SLICES-PLUS [23]	Graph and symmetry symbols	D	–	–	✓	Fixed encoder	Graph reconstruction
CrystalFormer [24]	Wyckoff sites and geometry	M	–	✓	✓	Fixed fields	Symmetry expansion
WyFormer [28]	Decorated Wyckoff tuples	D	–	–	✓	Fixed tuples	Prototype; geometry added later
PLaID++ [25]	Text over Wyckoff fields	D	–	✓	✓	Reused language tokenizer	Parsing and symmetry expansion
Matra-Genoa [26]	Discrete fields and geometry	M	–	✓	✓	Fixed fields	Mixed-field decoding

N refers to the structural latent encoder, rather than embeddings in the consuming model. G requires both lattice and sufficient positional variables; S can be present without G. The DiffCSP row describes its explicit state representation, and the CDVAE row its encoded latent. A dash marks absence in the selected representation.

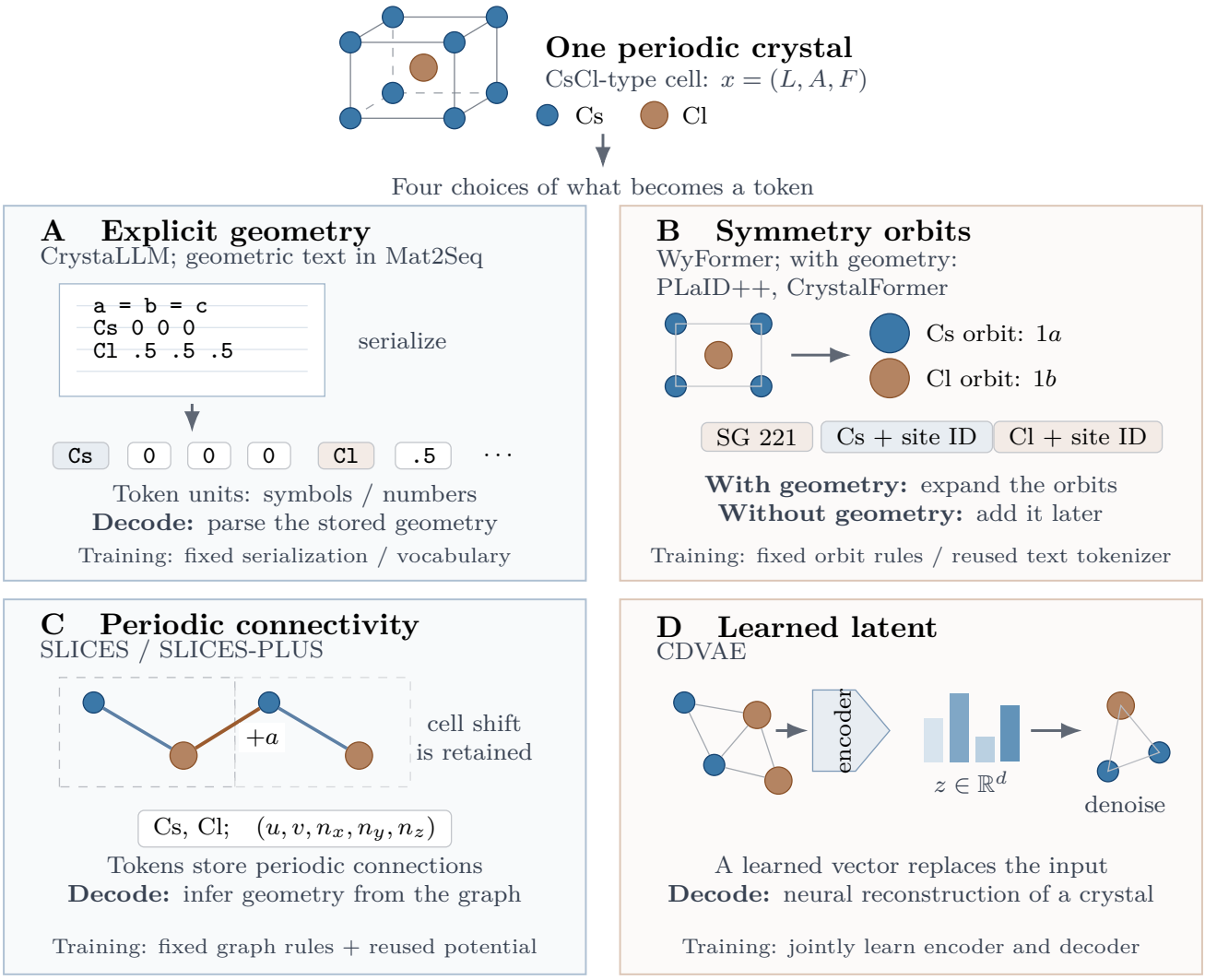


Figure 2: Four representative tokenization families illustrated from a CsCl-type periodic structure. Colored circles distinguish elements; graph sketches are schematic, and chips summarize fields rather than literal model token streams. Full geometry, orbit decorations, periodic connectivity, and learned latents retain different information and require different decoding algorithms.

4.4 Biomedical Observations

Draft suggestion: This family moves from molecular measurements to spatial morphology and clinical context. Keep the three object subsubsections distinct because their samples, relations, missing-data processes, and validation protocols differ. Use the modality section for image, table, graph, sequence, and signal methods.

4.4.1 Gene Expression Profiles

Draft suggestion: Representative work: Cui et al. [30]. Compare genes, expression values, ranked gene lists, expression bins, cells, and learned cell states. Discuss assay conditions, sparsity, batch effects, perturbations, and transfer across tissues or studies. Distinguish gene tokens from cell-level sample tokens.

4.4.2 Cell and Tissue Morphology

Draft suggestion: Discuss microscopy and digital pathology through pixels, patches, segmented cells, subcellular structures, tissue regions, and spatial neighborhoods. Compare token scale against rare-cell retention, tissue boundaries, spatial context, and cross-instrument transfer. Refer multimodal or perturbation systems that combine morphology with molecular measurements to Section 5.

4.4.3 Clinical Records and Measurements

Draft suggestion: Compare tokenization of clinical text, codes, laboratory measurements, physiological signals, medical images, and longitudinal events. Focus on missingness, irregular time, coding systems, patient context, privacy constraints, and validation across clinical sites. Keep modality-specific mechanics in Section 3.

4.5 Physical and Robotic Objects

Draft suggestion: This family covers physical variables, represented environments, and control outputs. Keep fields, scenes, and actions separate because they differ in their coordinate systems, decoding targets, and evaluation. Discuss a complete action-conditioned model of observations and transitions in Section 5.

4.5.1 Physical Fields

Draft suggestion: Representative work: Wu et al. [31]. Build on image, graph, geometry, and time-series methods from Section 3. Focus on physical variables, units, boundary conditions, conservation laws, discretization error, numerical stability, and transfer across grids or meshes.

4.5.2 Scenes and Geographical Regions

Draft suggestion: Representative work: Lam et al. [32]. Compare scene regions, objects, map cells, geographic regions, and geospatial graph nodes. Distinguish generic 3D geometry from location-specific coordinates, map projections, vertical levels, observation networks, and transfer across locations or seasons. Move joint state, action, and transition tokenization to Section 5.

4.5.3 Robot Actions

Draft suggestion: Representative work: Pertsch et al. [33]. Compare scalar controls, transformed coefficients, discrete actions, action chunks, and reusable action segments. Evaluate action reconstruction, control error, temporal abstraction, and closed-loop behavior. Exclude systems whose main contribution is joint observation, latent-state, action, and transition tokenization; discuss them as world models in Section 5.

4.6 Recommendation Data

Draft suggestion: Use this family for tokenization that represents recommendable items and observed interaction histories. Keep its vocabulary aligned with recommender-system research. Treat recommendation as the application context and place the concrete represented objects in the subsubsection below.

Draft suggestion: Begin with generative retrieval with semantic identifiers. Compare item identifiers, semantic identifiers, item sequences, sessions, and temporal histories. Ask whether the tokenizer preserves item identity, item coverage, interaction order, new-item handling, and decoding to valid items.

5 Tokenization for Complex Systems

Draft suggestion: Treat Sections 3 and 4 as the component library for this section. Include a system here only when its tokenizer combines components across modalities, domain objects, biological scales, or temporally coupled observations and actions. Multiple representation labels alone are insufficient. Do not repeat the construction of each component tokenizer. Study instead how components are aligned, kept separate, assigned token rates, related across scales or time, decoded jointly, and tested as one system.

5.1 Multimodal Language Models

Draft suggestion: Representative work: Lu et al. [34]. This subsection builds directly on the modality-specific tokenizers in Section 3. Compare separate, aligned, and shared token spaces through cross-modal correspondence, token-rate balance, attention allocation, and decoding support. Focus on models that combine language with images, video, audio, or other represented inputs and outputs. Distinguish this setting from biomolecular complexes, cellular perturbation systems, and world models, whose system structure follows scientific components or temporal transitions.

5.2 Biomolecular Complexes

Draft suggestion: Representative work: Abramson et al. [35]. Build on the nucleic-acid, protein, peptide, and small-molecule tokenizers in Section 4. Study how one system preserves component identity while representing cross-component contacts, stoichiometry, geometry, ions, ligands, and missing structures. Do not repeat each component tokenizer. Compare instead how the system aligns, relates, and jointly decodes several biomolecular objects.

5.3 Cellular Perturbation Systems

Draft suggestion: Representative starting point: Cui et al. [30]. Build on gene expression profiles and cell morphology in Section 4. Study how a tokenizer relates molecular interventions, cell state, time, dose, morphology, and measured response. Distinguish this subsection from biomolecular complexes, which model interacting components within an assembly, and from world models, which center on action-conditioned transitions in an environment. Compare how systems represent perturbations, preserve cell identity, align assays, and decode predicted responses.

5.4 World Models

[mengzhao]

Kickoff outline

- Scope: explain how world models jointly represent observations, latent states, actions, and transitions; connect component tokenizers to temporal prediction and interaction.
- Representation choices: compare discrete, continuous, and hybrid visual-latent, state, and action representations, focusing on token units, temporal grouping, and decoding targets.
- Literature organization: start from Genie [36] and group further references by representation and conditioning strategy after checking their tokenizer details.
- Comparison and evaluation: examine sequence compression, temporal consistency, action conditioning, rollout fidelity, and controllability; separate tokenizer-level evidence from whole-system results.
- Planned synthesis: summarize trade-offs among compression, reconstruction, prediction, and control, then identify open questions about temporal abstraction and transfer. Keep action-only methods in the domain-object section.

Draft suggestion: Representative work: Bruce et al. [36]. Study systems that jointly tokenize observations, latent states, actions, and transitions. Distinguish them from Section 4, which treats scenes and robot actions as separate domain objects, and from multimodal language models, which need not model action-conditioned change. Include action-only methods such as FAST in the domain-object section and cite them here only when comparing the action component of a complete world model. Evaluate temporal identity, rollout stability, controllability, and closed-loop behavior at the system level.

5.5 User Behavior Systems

6 Tokenizer Design and Evaluation

Draft suggestion: Connect every proposed property to a design choice and a measurable test. Separate properties of the tokenizer from outcomes of the downstream foundation model. Use the companion catalog to select representative systems, and move full record-level statistics to supplementary material after human review.

6.1 Core Properties of a Tokenizer System

Draft suggestion: Define a short set of properties that applies across modalities: input coverage, stable encoding, declared reconstruction, token rate, and compatibility with the consuming model. State which properties are required and which depend on task or data. Treat exact reconstruction as a requirement only when the declared representation is reversible.
https://arxiv.org/pdf/2511.11758#page=7.48: GeoBPE compression, reconstruction, generalization, interpretability

6.2 Task and Scientific Design Principles

Draft suggestion: Organize this subsection around prediction, generation, unified prediction and generation, and scientific data. For prediction, test discriminative information, generalization, and multiple scales. For generation, test reconstructability, compositional control, validity, and token rate. For science, test units, relations, symmetry, causality, measurement conditions, and response under intervention.

6.3 Benchmarks and Comparisons

Draft suggestion: Evaluate three linked outcomes: representation and system cost, reconstruction and scientific validity, and downstream utility. Match data, backbone, optimization, and compute where the claim concerns tokenization. Report tokenizer-training data separately, include repeated runs for learned tokenizers, and test rare cases, valid transformations, and realistic distribution shifts.

7 Future Directions

Draft suggestion: Treat foundation-model construction as the end goal while separating tokenizer-specific evidence from model-dependent evidence. Organize the research agenda around three questions: how tokenizers should be evaluated with and without foundation models, what foundation models require from tokenizers, and how the two systems should be designed together. For each direction, state a testable claim, a controlled comparison, and the conditions under which a simpler tokenizer remains preferable.

7.1 Evaluating Tokenizers with and without Foundation Models

Draft suggestion: Representative foundations: rate–distortion theory [37], VQ-VAE [2], and the scaling analysis of Byte Latent Transformer [38]. Organize evaluation around compression, intelligence, and their joint scaling. Without a foundation model, measure input coverage, encoding stability, token rate, vocabulary or codebook use, reconstruction under the declared equivalence, and the cost of training, encoding, and decoding. Report lossless rate when exact recovery is required, and report distortion together with structural or scientific validity for lossy tokenizers. With a foundation model, test whether the retained information supports prediction, generation, simulation, and control under matched data, backbone, optimization, parameter count, context budget, and total compute. Connect the two levels through rate–distortion–utility curves across model and data scales. Use raw bytes, pixels, residues, atoms, or observations as common denominators because token counts vary across tokenizers. Include reuse as a model-conditioned test: compatibility asks whether models can consume the same token meanings; transferability asks whether a tokenizer remains useful on new data or tasks; replaceability asks how much adaptation is required to substitute one tokenizer for another. Separate tokenizer-training and retokenization costs from foundation-model training, and do not attribute a full-system gain to tokenization without a controlled comparison.

7.2 Designing Tokenizers for Foundation Models

Draft suggestion: Representative work: Lu et al. [34] and Pertsch et al. [33]. Derive tokenizer requirements from the foundation model that consumes its output. Ask how the learning objective, attention pattern, context length, positional representation, output decoder, and deployment budget constrain unit boundaries, token rate, vocabulary or codebook size, and relation encoding. Prediction requires discriminative information and generalization across valid changes. Generation requires sufficient information for reconstruction, valid composition, and controllable output length. Simulation and action further require temporal identity, causal order, and stable closed-loop decoding. Study whether one tokenizer can support these uses or whether shared primitive units should feed task-specific compositions. Require deterministic encoding when models or checkpoints must share token identifiers. Test whether the same tokenizer can serve different foundation-model sizes and architectures, and state the adaptation required when it cannot. For scientific foundation models, add object identity, symmetry, conservation, measurement conditions, and scale as explicit design constraints rather than inferring them from downstream accuracy.

7.3 Co-designing Tokenizers and Foundation Models

Draft suggestion: Representative work: Ryoo et al. [39], Pagnoni et al. [38], and Lu et al. [34]. Reverse the usual design question and ask how the foundation model should change when the tokenizer returns variable-length, overlapping, hierarchical, graph-structured, continuous, or stochastic units. Study positional and relation encodings that respect token geometry and hierarchy; attention and routing that allocate computation according to token scale, uncertainty, or information content; and training objectives that evaluate predictions in the original data space as well as the token space. A model should retain token boundaries, parent–child relations, and decoding uncertainty when these quantities affect the task. Joint training should control codebook collapse, unstable boundaries, and shifts in token meaning, while preserving a clear encoding and decoding procedure. Compare fixed-token backbones with architectures designed for the same tokenizer under matched total compute. Extend checkpoint reuse through token mappings, embedding adaptation, or partial retraining, and report the performance recovery cost. Treat algorithmic tokenizers as valid co-design choices: co-design concerns coordinated assumptions between modules and does not require an end-to-end neural tokenizer.

8 Conclusion

Draft suggestion: End the final manuscript with the strongest claim supported by the completed evidence, followed by one clear limitation and one open research question. Remove claims that depend only on unmatched full-model comparisons or unverified tokenizer details.

References

[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[2] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

[3] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 1715–1725, 2016.

[4] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in neural information processing systems*, 33: 12449–12460, 2020.

[5] Mikhail Galkin, Etienne Denis, Jiapeng Wu, and William L Hamilton. Nodepiece: Compositional and parameter-efficient representations of large knowledge graphs. *arXiv preprint arXiv:2106.12144*, 2021.

[6] Xumin Yu, Lulu Tang, Yongming Rao, Tiejun Huang, Jie Zhou, and Jiwen Lu. Point-bert: Pre-training 3d point cloud transformers with masked point modeling. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19291–19300. IEEE, 2022.

[7] Jessica Lin, Eamonn Keogh, Stefano Lonardi, and Bill Chiu. A symbolic representation of time series, with implications for streaming algorithms. In *Proceedings of the 8th ACM SIGMOD workshop on Research issues in data mining and knowledge discovery*, pages 2–11, 2003.

[8] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P De Almeida, Hassan Sirelkhatim, et al. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature methods*, 22(2):287–297, 2025.

[9] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C Lawrence Zitnick, Jerry Ma, et al. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the national academy of sciences*, 118(15): e2016239118, 2021.

[10] Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *International conference on machine learning*, pages 2323–2332. PMLR, 2018.

[11] Tzyy-Shyang Lin, Connor W. Coley, Hidenobu Mochigase, Haley K. Beech, Wencong Wang, Zi Wang, Eliot Woods, Stephen L. Craig, Jeremiah A. Johnson, Julia A. Kalow, Klavs F. Jensen, and Bradley D. Olsen. BigSMILES: A structurally-based line notation for describing macromolecules. *ACS Central Science*, 5(9): 1523–1531, 2019. doi: 10.1021/acscentsci.9b00476.

[12] Matteo Aldeghi and Connor W. Coley. A graph representation of molecular ensembles for polymer property prediction. *Chemical Science*, 13(35):10486–10498, 2022. doi: 10.1039/D2SC02839E.

[13] Christopher Kuenneth and Rampi Ramprasad. polyBERT: a chemical language model to enable fully machine-driven ultrafast polymer informatics. *Nature Communications*, 14:4099, 2023. doi: 10.1038/s41467-023-39868-6.

[14] Changwen Xu, Yuyang Wang, and Amir Barati Farimani. TransPolymer: a transformer-based language model for polymer property predictions. *npj Computational Materials*, 9:64, 2023. doi: 10.1038/s41524-023-01016-5.

[15] Anagha Savit, Harikrishna Sahu, Shivank S. Shukla, Wei Xiong, and Rampi Ramprasad. polyBART: A chemical linguist for polymer property prediction and generative design. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12104–12119. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.647. URL https://aclanthology.org/2025.findings-emnlp.647/.

[16] Yihan Zhu, Gang Liu, Eric Inae, Tengfei Luo, and Meng Jiang. Learning repetition-invariant representations for polymer informatics. In *Advances in Neural Information Processing Systems*, volume 38, pages 88451–88470. Curran Associates, Inc., 2025. doi: 10.52202/085713-2957. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/7fe3921147c968d0b57a224c0d07e21d-Abstract-Conference.html.

[17] Tian Xie and Jeffrey C. Grossman. Crystal graph convolutional neural networks for an accurate and interpretable prediction of material properties. *Physical Review Letters*, 120(14):145301, 2018. doi: 10.1103/PhysRevLett.120.145301. URL https://doi.org/10.1103/PhysRevLett.120.145301.

[18] Robin Ruff, Patrick Reiser, Jan Stühmer, and Pascal Friederich. Connectivity optimized nested line graph networks for crystal structures. *Digital Discovery*, 3(3):594–601, 2024. doi: 10.1039/D4DD00018H. URL https://doi.org/10.1039/D4DD00018H.

[19] Rui Jiao, Wenbing Huang, Peijia Lin, Jiaqi Han, Pin Chen, Yutong Lu, and Yang Liu. Crystal structure prediction by joint equivariant diffusion. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0767. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/38b787fc530d0b31825827e2cc306656-Abstract.html.

[20] Luis M. Antunes, Keith T. Butler, and Ricardo Grau-Crespo. Crystal structure generation with autoregressive large language modeling. *Nature Communications*, 15, 2024. doi: 10.1038/s41467-024-54639-7. URL https://doi.org/10.1038/s41467-024-54639-7.

[21] Keqiang Yan, Xiner Li, Hongyi Ling, Kenna Ashen, Carl Edwards, Raymundo Arróyave, Marinka Zitnik, Heng Ji, Xiaofeng Qian, Xiaoning Qian, and Shuiwang Ji. Invariant tokenization of crystalline materials for language model enabled generation. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-3971. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/e23133d34964a0a09f6d076fc4b922a4-Abstract-Conference.html.

[22] Hang Xiao, Rong Li, Xiaoyang Shi, Yan Chen, Liangliang Zhu, Xi Chen, and Lei Wang. An invertible, invariant crystal representation for inverse design of solid-state materials using generative deep learning. *Nature Communications*, 14:7027, 2023. doi: 10.1038/s41467-023-42870-7. URL https://doi.org/10.1038/s41467-023-42870-7.

[23] Baoning Wang, Zhiyuan Xu, Zhiyu Han, Qiwen Nie, Xi Chen, Hang Xiao, and Gang Yan. SLICES-PLUS: A crystal representation leveraging spatial symmetry. *Materials & Design*, 253, 2025. doi: 10.1016/j.matdes.2025.113856. URL https://doi.org/10.1016/j.matdes.2025.113856.

[24] Zhendong Cao, Xiaoshan Luo, Jian Lv, and Lei Wang. Space group informed transformer for crystalline materials generation. *Science Bulletin*, 70(21):3522–3533, 2025. doi: 10.1016/j.scib.2025.09.035. URL https://doi.org/10.1016/j.scib.2025.09.035.

[25] Andy Xu, Rohan Desai, Larry Wang, Ethan T. Ritz, and Gabriel Hope. PLaID++: A preference aligned language model for targeted inorganic materials design. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 140903–140922. PMLR, 2026. URL https://proceedings.mlr.press/v306/xu26l.html.

[26] Pierre-Paul De Breuck, Hashim A. Piracha, Gian-Marco Rignanese, and Miguel A. L. Marques. A generative material transformer using wyckoff representation. *npj Computational Materials*, 2026. doi: 10.1038/s41524-025-01940-8. URL https://doi.org/10.1038/s41524-025-01940-8.

[27] Rhys E. A. Goodall, Abhijith S. Parackal, Felix A. Faber, Rickard Armiento, and Alpha A. Lee. Rapid discovery of stable materials by coordinate-free coarse graining. *Science Advances*, 8(30):eabn4117, 2022. doi: 10.1126/sciadv.abn4117. URL https://doi.org/10.1126/sciadv.abn4117.

[28] Nikita Kazeev, Wei Nong, Ignat Romanov, Ruiming Zhu, Andrey E. Ustyuzhanin, Shuya Yamazaki, and Kedar Hippalgaonkar. Wyckoff transformer: Generation of symmetric crystals. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 29495–29526. PMLR, 2025. URL https://proceedings.mlr.press/v267/kazeev25a.html.

[29] Tian Xie, Xiang Fu, Octavian-Eugen Ganea, Regina Barzilay, and Tommi Jaakkola. Crystal diffusion variational autoencoder for periodic material generation. In *International Conference on Learning Representations*, 2022. URL https://arxiv.org/abs/2110.06197.

[30] Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scgpt: toward building a foundation model for single-cell multi-omics using generative ai. *Nature methods*, 21(8): 1470–1480, 2024.

[31] Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for pdes on general geometries. *arXiv preprint arXiv:2402.02366*, 2024.

[32] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, et al. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023.

[33] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.

[34] Jiasen Lu, Christopher Clark, Sangho Lee, Zichen Zhang, Savya Khosla, Ryan Marten, Derek Hoiem, and Aniruddha Kembhavi. Unified-io 2: Scaling autoregressive multimodal models with vision, language, audio, and action. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26429–26445. IEEE, 2024.

[35] Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J Ballard, Joshua Bambrick, et al. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630(8016):493–500, 2024.

[36] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first international conference on machine learning*, 2024.

[37] Claude Elwood Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27 (3):379–423, 1948.

[38] Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, 2025.

[39] Michael Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Tokenlearner: Adaptive space-time tokenization for videos. *Advances in neural information processing systems*, 34:12786–12797, 2021.